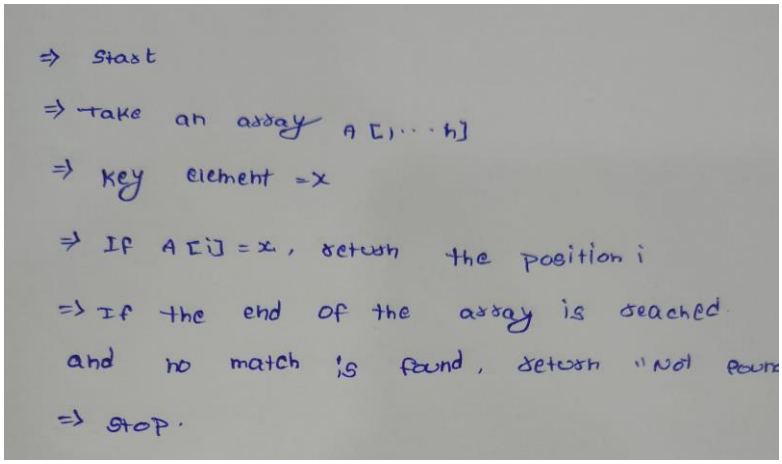
PRACTICAL-02

AIM: To implementation and time analysis of linear and binary search algorithm.

1. LINEAR SEARCH

ALGORITHM:



PROGRAM:

```
#include      <iostream>
using namespace std;

int main() {
   int arr[100],i,n,key;
   cout<<"enter number of array elements:"<<endl; cin>>n;
   cout<<"enter array elements:"<<endl;
   for(i=0;i<n;i++){
     cin>>arr[i];

   }
   cout<<"enter element to serach:"<<endl;
   cin>>key;
   for(i=0;i<n;i++){
```

```
    if(arr[i]==key)
    {
        cout<<"element found at index of:"<<i;
    }

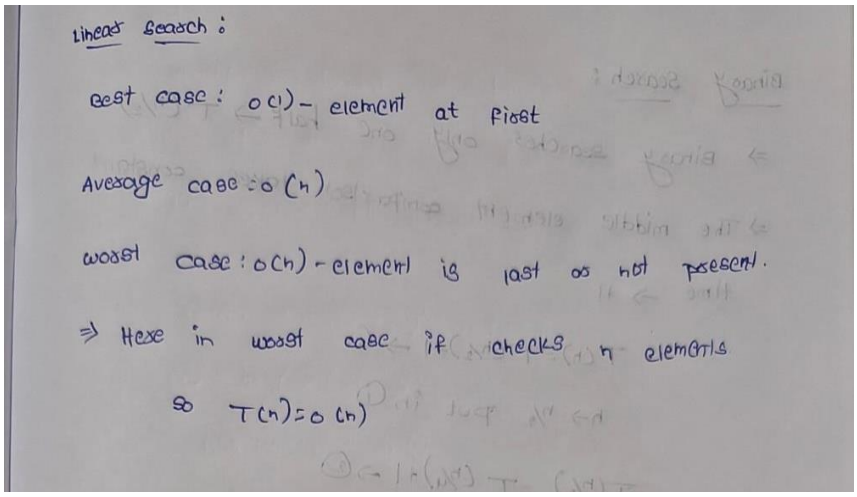
    }

    return 0;
}
```

OUTPUT:

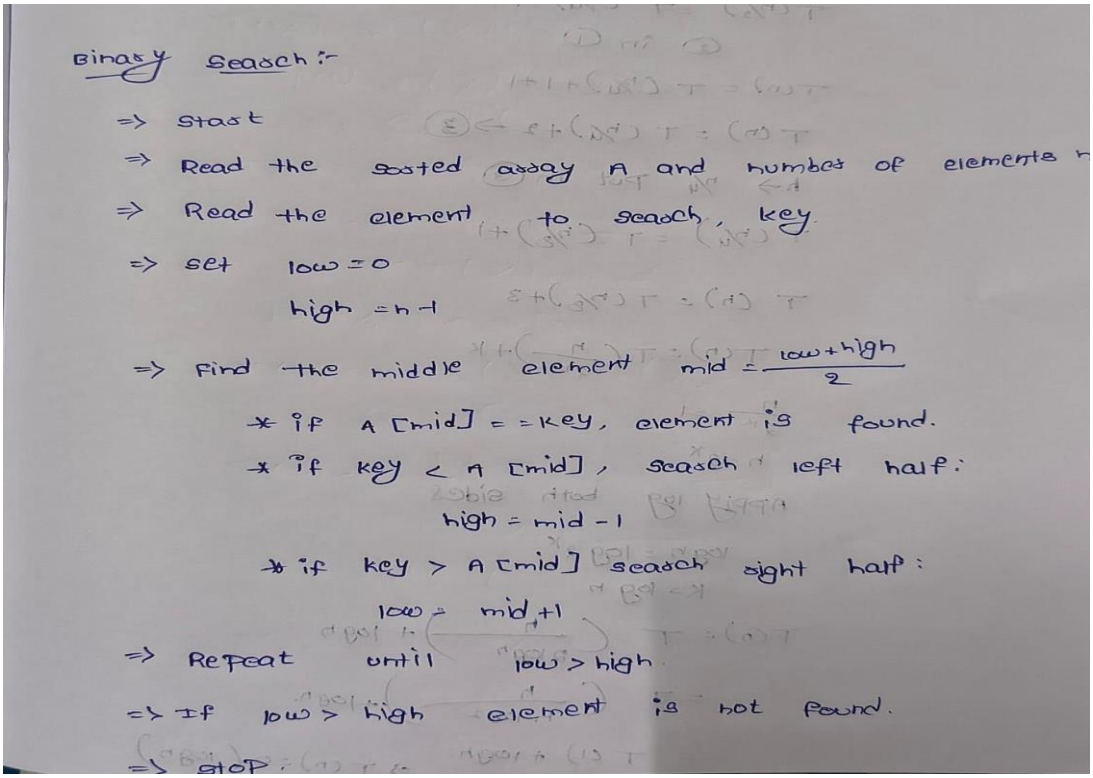
```
enter number of array elements:
5
enter array elements:
7
9
5
34
78
enter element to serach:
5
element found at index of:2
```

TIME COMPLEXITY:



2. BINARY SEARCH

ALGORITHM:



PROGRAM:

```
#include <iostream>
using namespace std;

void bubblesort(int arr[], int n) {
   for(int i=0; i<n-1; i++) {
      for(int j=0; j<n-i-1; j++) {
         if(arr[j] > arr[j+1]) {
            int temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
```

```
        }
      }
    }
}
int binarySearch(int arr[], int n, int key) { int
    low = 0, high = n-1;
    while(low <= high) {
        int mid = (low + high) / 2;
        if(arr[mid] == key) return mid;
        else if(arr[mid] < key) low = mid+1;
        else high = mid-1;
    }
    return -1;
}

int main() {
    int arr[100], n, key;
    cout << "Enter number of array elements: "; cin
    >> n;

    cout << "Enter array elements: ";
    for(int i=0; i<n; i++) {
        cin >> arr[i];
    }

    bubblesort(arr, n);

    cout << "Sorted elements: ";
```

```
    for(int i=0; i<n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
    cout << "Enter element to search: "; cin
    >> key;

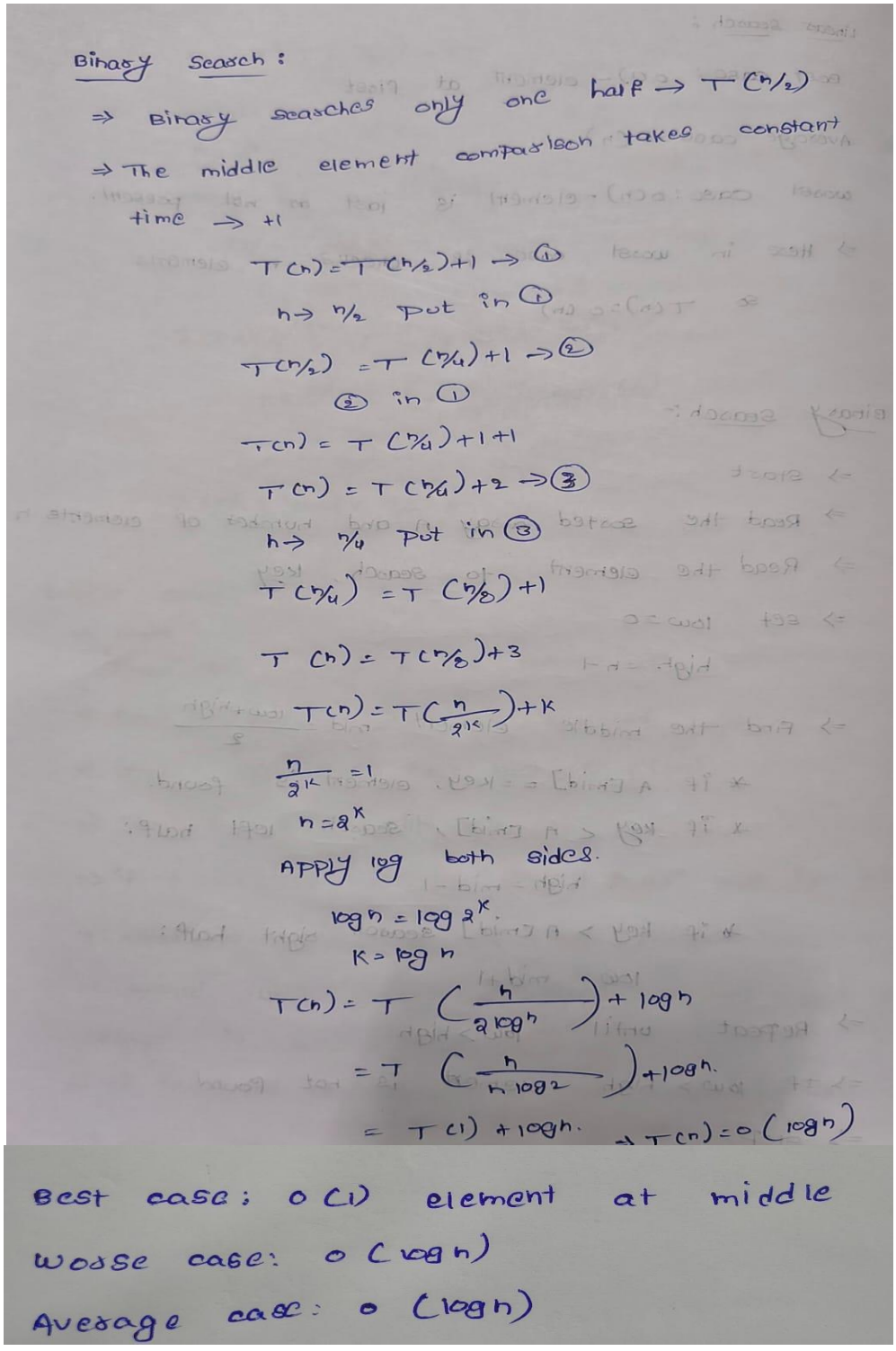
    int result = binarySearch(arr, n, key);
    if(result != -1)
        cout << "Element found at index " << result << endl;
    else
        cout << "Element not found" << endl;

    return 0;
}
```

OUTPUT:

```
Enter number of array elements: 6
Enter array elements: 8
4
6
1
9
19
Sorted elements: 1 4 6 8 9 19
Enter element to search: 6
Element found at index 2
```

TIME COMPLEXITY:



CONCLUSION:

Linear Search is simple and works on unsorted arrays but is slow with $O(n)$time. Binary Search is much faster with $O(\log n)$but requires the array to be sorted. For sorting, Bubble and Selection are basic $O(n^2)$, Insertion improves to $O(n)$in best case, Quick Sort is efficient on average, and Merge Sort is consistently $O(n \log n)$though it needs extra space.